DOC-SCEPTIC

Agent RAG d’aide à la décision d’architecture Microsoft Fabric

Rapport technique détaillé du projet

25 / 25	0	100 %
cas réussis	erreur API	benchmark interne v1

Python • LangGraph • Supabase / pgvector • Streamlit • Reranking • LLM

Résumé exécutif

Doc-Sceptic est un assistant RAG spécialisé dans l'aide à la décision d'architecture Microsoft Fabric. Le projet a été conçu pour répondre à un problème simple : une réponse générée par un LLM peut paraître techniquement convaincante sans être suffisamment soutenue par la documentation. Doc-Sceptic introduit donc une chaîne de contrôle entre la question utilisateur et la génération finale.

L'agent analyse les contraintes du besoin, recherche des passages dans une documentation indexée, réévalue leur pertinence, vérifie la couverture documentaire puis choisit entre trois comportements principaux : recommander, demander une clarification ou s'abstenir. Lorsqu'une réponse est autorisée, les affirmations techniques sont accompagnées de citations normalisées [S1], [S2], etc.

La version finale comprend également une interface Streamlit, une authentification Supabase, un historique persistant des conversations, l'export et la suppression des échanges, ainsi qu'un mode conversationnel qui évite de déclencher inutilement le RAG sur les salutations et le small talk.

Le benchmark fonctionnel interne v1 contient 25 scénarios. La dernière exécution complète a obtenu 25/25 cas réussis et 0 erreur API. Ce résultat mesure uniquement les scénarios du benchmark interne et ne doit pas être interprété comme une précision générale de 100 % du système.

Dimension	État final
Objectif	Décider sur la base de preuves documentaires
Orchestration	LangGraph
Recherche	Supabase + pgvector, embeddings et reranking
UI	Streamlit avec authentification et historique
Décisions	Recommend / Clarify / Abstain
Évaluation	Benchmark fonctionnel interne v1 : 25/25, 0 erreur API

1. Contexte et problématique

1.1 Problème traité

Dans un contexte d'architecture data, le choix entre Warehouse, Lakehouse, Data Factory, SQL Database, Data Engineering ou Power BI dépend du workload réel et de contraintes comme le type de données, le style de développement, les transactions multi-tables, l'usage de Spark, Power BI ou le streaming.

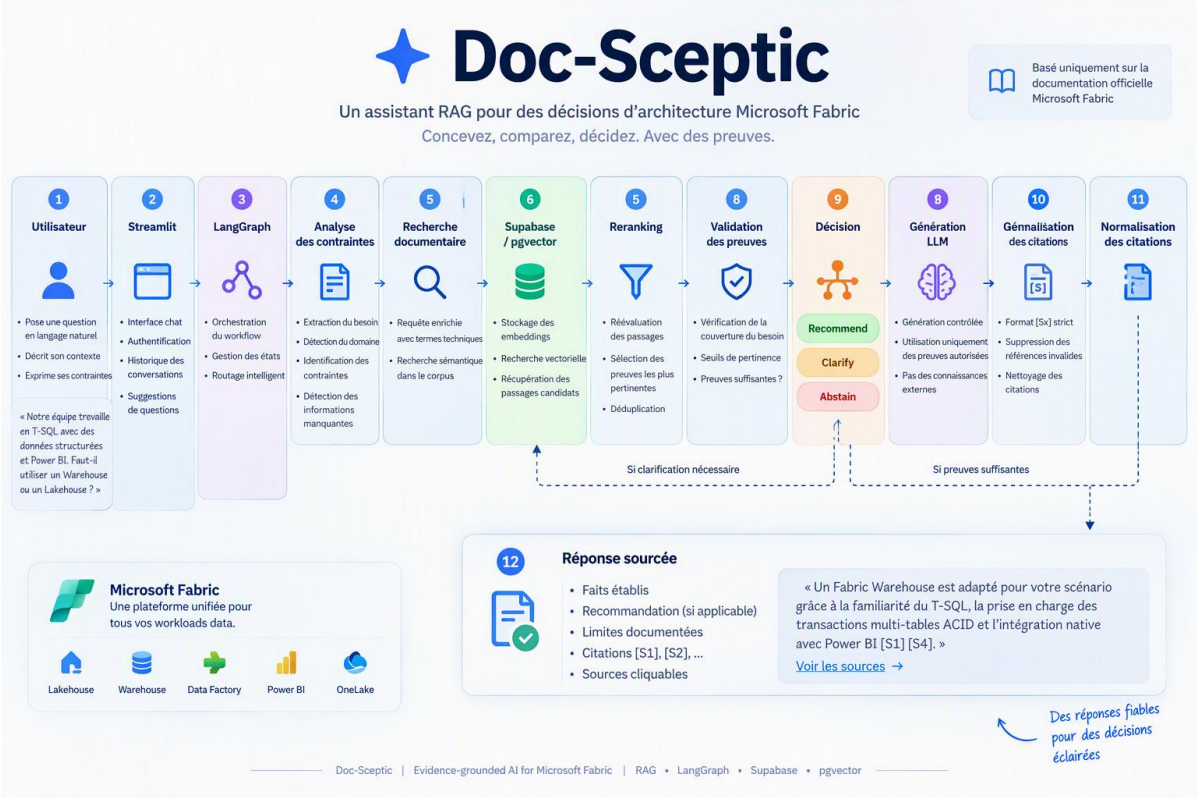
Le risque d'un chatbot classique est de transformer une question sous-spécifiée en recommandation trop affirmative. Doc-Sceptic a donc été conçu autour d'une idée : la génération n'est autorisée qu'après analyse des contraintes et validation des preuves.

1.2 Objectifs

Comprendre le besoin exprimé en langage naturel ; retrouver les passages documentaires pertinents ; mesurer si les preuves couvrent réellement les contraintes ; recommander uniquement lorsque cette couverture est suffisante ; demander des précisions si le besoin est ambigu ; s'abstenir dans les cas non démontrables ; conserver l'historique et les sources dans une application utilisable.

2. Architecture fonctionnelle

2.1 Chaîne de decision



Utilisateur → Streamlit → LangGraph → analyse des contraintes → recherche documentaire → Supabase / pgvector → reranking → validation des preuves → Recommend / Clarify / Abstain → génération LLM si nécessaire → normalisation des citations → réponse sourcée.

2.2 Nœuds LangGraph

Le graphe est structuré autour des nœuds analyze, chat, clarify, retrieve, assess, generate et abstain. Le rôle de ce découpage est d'éviter le chemin simpliste question → LLM → réponse et de rendre la décision observable.

2.3 Séparation conversation / connaissance

Les salutations, remerciements, demandes d'identité ou d'aide sont traités localement comme action=chat. Ils ne déclenchent pas Supabase et ne produisent pas de citations. Les questions techniques suivent le pipeline RAG.

3. Analyse des contraintes

3.1 Détection du domaine

Le système identifie notamment les domaines data_factory, medallion, warehouse_lakehouse, data_engineering, power_bi et fabric_general. Des règles de priorité ont été ajoutées pour éviter les mauvais routages, par exemple sur les cas OLTP ou les scénarios hybrides Warehouse/Lakehouse.

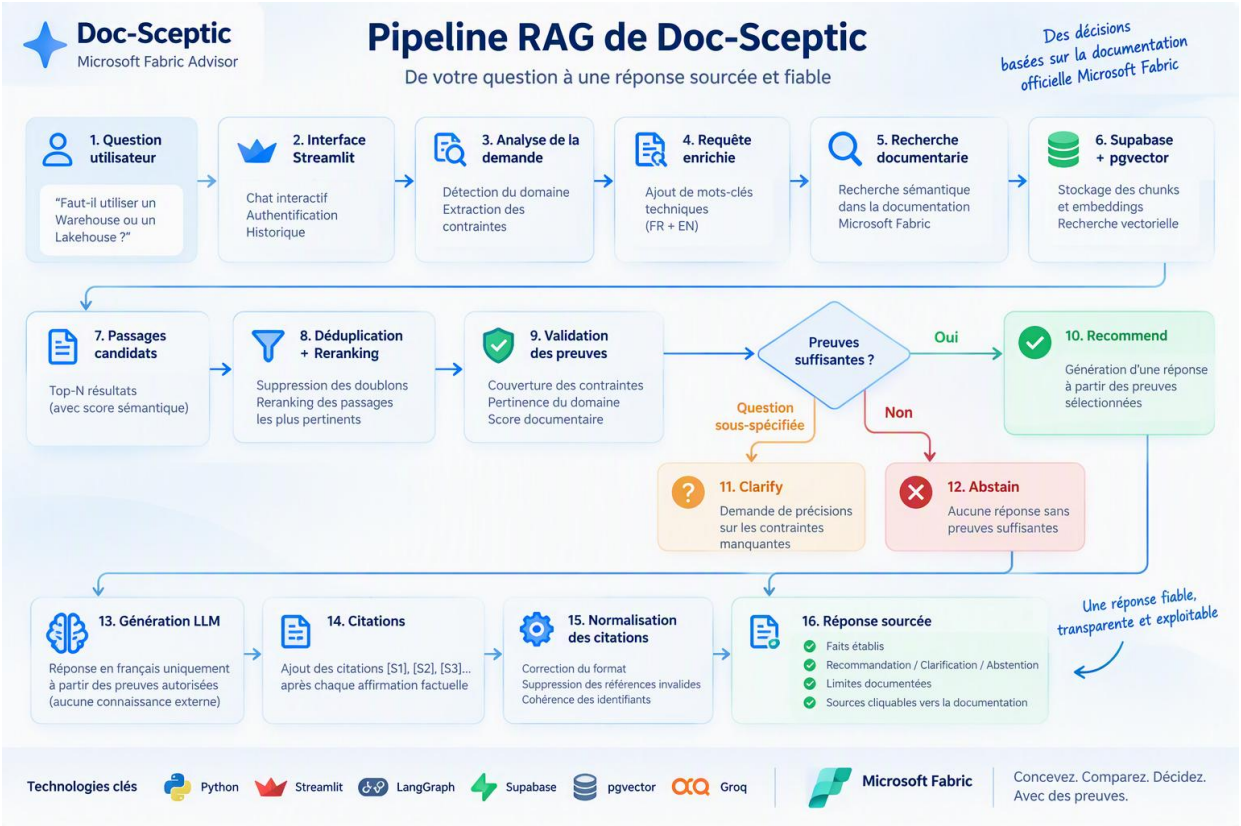
3.2 Contraintes extraites

Les structures de Requirements contiennent notamment data_types, development_styles, multi_table_transactions, power_bi, streaming, unstructured_data et missing_constraints. Ces éléments servent ensuite à enrichir la requête et à valider la couverture des preuves.

3.3 Clarification

Une question telle que « Je démarre un projet Microsoft Fabric. Warehouse ou Lakehouse ? » est volontairement traitée comme insuffisamment spécifiée. L'agent demande alors le type de données, le style SQL ou Spark, le besoin de transactions multi-tables et l'usage analytique ou Power BI.

4. Pipeline RAG



4.1 Corpus et indexation

La documentation technique est découpée en chunks, enrichie de métadonnées puis vectorisée. Les embeddings sont stockés dans Supabase/PostgreSQL via pgvector afin de permettre la recherche sémantique.

4.2 Enrichissement de requête

La requête conserve la question originale mais ajoute du vocabulaire documentaire en anglais. Exemple : T-SQL, Warehouse, full multi-table ACID transactions, Apache Spark, Direct Lake ou SQL analytics endpoint selon les contraintes détectées.

4.3 Retrieval, déduplication et reranking

Plus de candidats que le top-k final sont récupérés afin de laisser une chance aux passages littéraux pertinents. Les chunks identiques ou quasi identiques sont dédupliqués, puis les preuves sont classées par score après reranking.

4.4 Validation des preuves

La fonction assess vérifie le meilleur score, la couverture des signaux requis, la présence d'un document correspondant au domaine et la disponibilité d'une preuve transactionnelle lorsqu'elle est nécessaire. Une preuve simplement proche sémantiquement ne suffit donc pas toujours.

5. Stratégies Recommend, Clarify et Abstain

5.1 Recommend

Lorsque les preuves sont suffisantes, l'action finale devient recommend. Le LLM reçoit uniquement les preuves sélectionnées et doit produire trois parties : Faits établis, Recommandation et Limites documentées.

5.2 Clarify

Si le besoin manque de contraintes déterminantes, l'agent n'effectue pas de choix artificiel et demande une précision. Cette branche protège le système contre les recommandations sur-spécifiées.

5.3 Abstain

Si les preuves sont insuffisantes ou si la demande exige une garantie impossible à démontrer, le système s'abstient. Des cas déterministes couvrent notamment le coût exact non documenté, la garantie de zéro panne et la latence exacte sans paramètres de workload.

6. Génération et citations

6.1 Prompt de génération

Le prompt interdit l'usage de connaissances externes aux preuves autorisées et demande une citation après les affirmations factuelles. Il interdit aussi d'inventer une capacité, une limitation, un coût, une performance, une garantie ou une procédure.

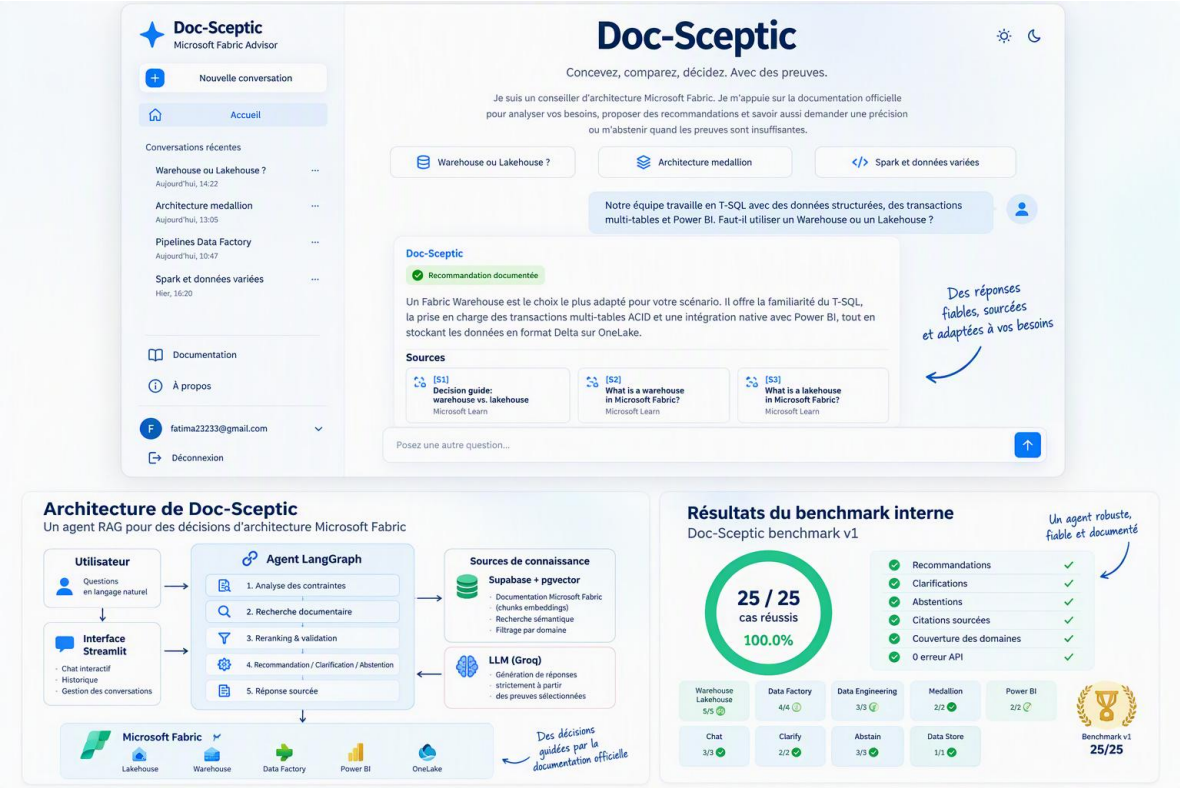
6.2 Normalisation des citations

Une étape déterministe corrige les formats dégradés observés durant les tests. Les formes S1], [S1]S2] ou T1][S1] peuvent être ramenées aux références autorisées [S1], [S2], etc. Les références hors plage sont supprimées.

6.3 Couverture de la question

Le prompt final exige de traiter tous les éléments explicitement demandés lorsque les preuves les couvrent. Cette règle a notamment été ajoutée après un cas Data Factory où la réponse traitait les connecteurs mais pas suffisamment l'authentification et le réseau.

7. Interface et persistence



7.1 Streamlit

L'application propose un chat, des suggestions de questions, des badges de décision, des cartes de sources et une interface bleu glacier. Le design final est orienté portfolio et démonstration.

7.2 Authentification et historique

Supabase assure l'authentification et la persistance. L'utilisateur peut créer un compte, se connecter, créer une conversation, retrouver une conversation, l'exporter, la supprimer et se déconnecter.

7.3 Historique persistant

L'historique est relu depuis le store Supabase à chaque rerun utile, ce qui évite de dépendre uniquement de st.session_state. La sidebar conserve aussi le bouton de réouverture Streamlit pour ne pas devenir inaccessible si elle est repliée.

8. Robustesse du client LLM

8.1 Réponses vides

Une difficulté importante a été la génération de réponses vides alors que les preuves étaient valides. Le diagnostic a mis en évidence des sorties interrompues par la limite de génération plutôt qu'un défaut du retrieval.

8.2 Gestion des erreurs temporaires

Le client LLM a été renforcé avec plusieurs tentatives, un budget de génération plus adapté, une gestion des erreurs temporaires et des réponses explicites lorsque la génération reste impossible.

8.3 Rate limits

La gestion du code 429 et du délai de retry permet au benchmark de distinguer les erreurs API des échecs fonctionnels. L'exécution finale de référence ne contient aucune erreur API.

9. Évaluation

9.1 Benchmark fonctionnel interne v1

Le benchmark comprend 25 scénarios couvrant Warehouse/Lakehouse, Data Engineering, Data Factory, Power BI, Medallion, Data Store, chat, clarification et abstention. Chaque cas contrôle plusieurs dimensions plutôt qu'un score unique.

9.2 Résultat final

La dernière exécution complète a produit : 25 cas demandés, 25 cas évalués, 25/25 tests réussis, 100,0 % sur ce benchmark interne et 0 erreur API. Les contrôles action, domain, choice, answer_terms, source_domain, sources_present et inline_citations obtiennent chacun 25/25.

9.3 Interprétation correcte

Le résultat 25/25 signifie que les 25 scénarios définis dans le benchmark fonctionnel interne v1 passent. Il ne constitue pas une preuve de précision générale de 100 % sur l'ensemble des questions Microsoft Fabric.

10. Difficultés et corrections majeures

10.1 Transactions multi-tables

Le retrieval initial pouvait récupérer des documents trop génériques. La correction a combiné enrichissement de requête, recherche plus large et validation explicite de la preuve Warehouse + transaction multi-table.

10.2 Data Factory

La couverture des connecteurs, de l'authentification et du réseau a nécessité de renforcer la règle de couverture de la question. Le benchmark a également été ajusté lorsque certaines attentes textuelles étaient trop rigides par rapport à une réponse pourtant correcte.

10.3 Medallion

Une question Bronze/Silver/Gold était initialement classée comme question vague à cause de l'expression « quelle architecture ». La logique a été corrigée afin de considérer le domaine Medallion comme déjà suffisamment explicite.

10.4 Citations et small talk

La phase de finition a corrigé deux défauts visibles : les citations mal formées et le small talk envoyé vers le RAG. Ces deux corrections ont été validées puis le benchmark complet est resté à 25/25.

11. Compétences démontrées

11.1 AI / RAG Engineering

Embeddings, vector database, retrieval, reranking, RAG, prompt grounding, validation des preuves, citations, orchestration agentique et gestion de l'abstention.

11.2 Data Engineering

Ingestion documentaire, chunking, métadonnées, stockage vectoriel, pipeline d'indexation, recherche, validation et intégration Supabase/PostgreSQL.

11.3 Engineering produit

Interface Streamlit, authentification, persistance des conversations, export, tests, benchmark, GitHub, documentation et démonstration vidéo.

12. Limites

12.1 Limites

Le comportement dépend du corpus indexé, de la qualité du chunking, des embeddings, du reranker, des seuils et du LLM. Le benchmark actuel mesure 25 scénarios définis et ne couvre pas l'ensemble des formulations ou évolutions de Microsoft Fabric.